

Real-Time Analytics Dashboard

Project Goal

Build a real-time analytics dashboard using **Node.js, Express, Redis, MongoDB, HyperLogLog, and Server-Sent Events (SSE)**.

The system should capture user activity, process analytics in Redis, store historical events in MongoDB, and display live analytics on a dashboard.

Version 1 — Supported Events

Start with **6 events**:

Event	Example	Useful For
<code>page_view</code>	User opens <code>/dashboard</code>	Page popularity and traffic
<code>login</code>	User logs in	Login activity and unique users
<code>logout</code>	User logs out	User activity
<code>file_upload</code>	User uploads a file	Feature usage
<code>file_download</code>	User downloads a file	Feature usage
<code>search</code>	User searches for something	Search activity

Event Ingestion

Use one event endpoint instead of creating a separate endpoint for every event.

```
POST /events
```

The event type determines what the backend does with the event.

Conceptually:

```
Frontend
  ↓
POST /events
  ↓
```

```
Express
  ↓
Validate event
  ↓
Process analytics
  ↓
Redis
  ↓
Store historical event
```

Event Structure

An event can contain:

```
eventType
page
metadata
```

The backend should handle information such as:

- `userId`
- `timestamp`
- request information where appropriate

Do not blindly trust sensitive information sent by the frontend.

Redis Responsibilities

Redis should be the **real-time analytics engine**.

Different Redis data structures should solve different analytics problems.

1. Counters

Use Redis counters for metrics such as:

- Total events
- Page views
- Uploads
- Downloads
- Logins

Example concept:

```
events:today
```

2. HyperLogLog

Use HyperLogLog for approximate unique-user counts.

Examples:

```
unique-users:today  
unique-users:this-hour
```

HyperLogLog answers:

Approximately how many unique users were recorded?

It does **not** tell you which users were recorded.

The benefit is very low memory usage compared with storing every user ID in a normal Set.

3. Sorted Sets

Use Redis Sorted Sets for rankings.

Examples:

- Most visited pages
- Most active users
- Most popular events

Conceptually:

```
/products    → 1,283  
/dashboard   → 932  
/home        → 821
```

The score represents the number of occurrences.

4. Pub/Sub

Redis Pub/Sub can be used for the real-time update pipeline.

Conceptually:

```
Event occurs
  ↓
Redis analytics updated
  ↓
Redis publishes notification
  ↓
SSE manager receives notification
  ↓
Dashboard receives update
```

Pub/Sub can be introduced after the basic analytics system works.

MongoDB Responsibilities

MongoDB should handle **historical event storage**.

Redis handles fast, current analytics while MongoDB provides longer-term event data.

An event record could contain:

```
userId
eventType
page
timestamp
metadata
```

This allows future queries such as:

Show me yesterday's activity.

Show me all file downloads from last week.

Analytics API

The dashboard needs endpoints for retrieving analytics.

Possible structure:

```
GET /analytics/overview
GET /analytics/events
GET /analytics/pages
GET /analytics/users
```

These do not all need to exist immediately.

Start with the minimum required endpoint and split the analytics API as the project grows.

Server-Sent Events

Use SSE for the real-time dashboard connection.

The browser establishes one connection:

```
Dashboard
  ↓
/analytics/stream
  ↓
SSE Manager
```

The connection stays open.

When analytics change:

```
Redis
  ↓
SSE Manager
  ↓
Browser
  ↓
Dashboard updates
```

This avoids repeatedly polling the server.

Dashboard

The dashboard should contain:

Overview Cards

Active Users

Unique Users

Total Events

Page Views

Example:

Active Users	127
Unique Users	3,421
Total Events	18,492
Page Views	12,839

Live Activity

Display recent events as they happen.

Example:

10:07:21 User viewed /products

10:07:20 User uploaded a file

10:07:19 User logged in

Popular Pages

Display the most visited pages.

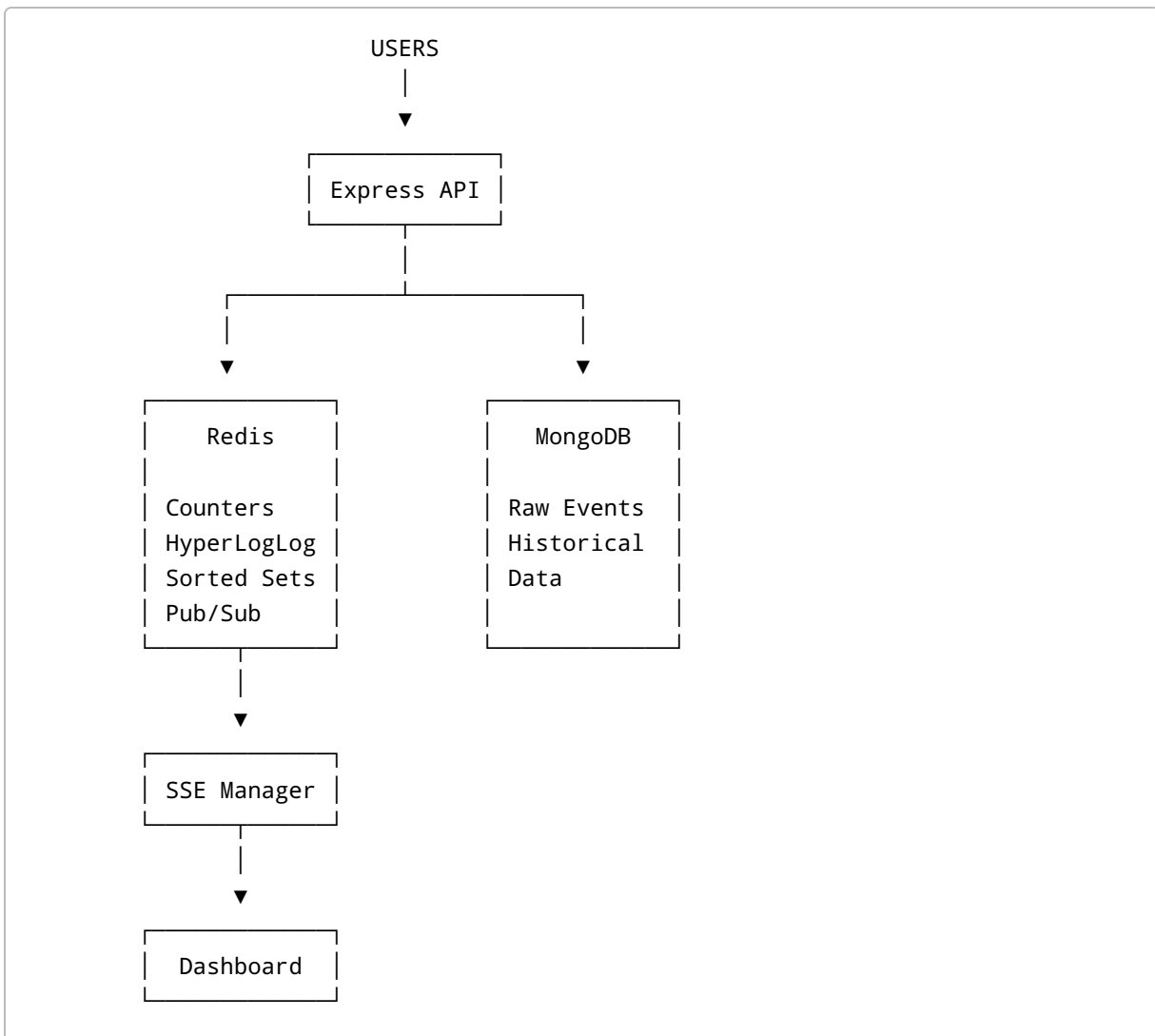
/products	1,283
/dashboard	932
/home	821

Event Graph

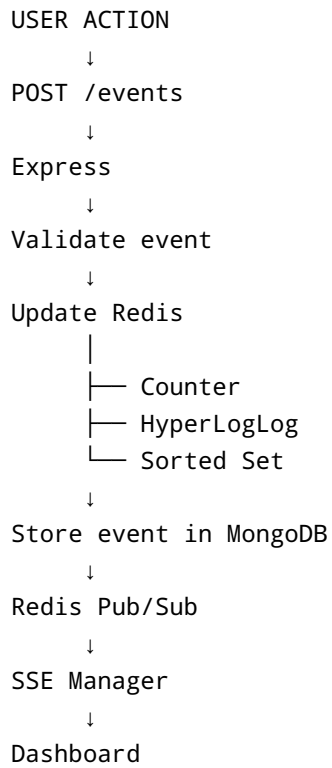
Display event activity over time.

The graph should update while the dashboard is open.

Architecture



Event Flow



Suggested Project Structure

```
project/
|
|-- controllers/
|
|-- routes/
|
|-- models/
|
|-- services/
|   |-- analytics/
|   |-- redis/
|   |-- sse/
|
|-- middleware/
|
```



```
├─ config/
├─ utils/
├─ public/
│   └─ dashboard/
└─ server.js
```

The important separation is:

```
HTTP logic
≠
Analytics logic
≠
Redis logic
≠
SSE logic
```

Development Phases

Phase 1 — Event Ingestion

Build:

- Event endpoint
- Event validation
- Supported event types

Phase 2 — Redis

Learn:

- Redis connection
- Strings
- Counters
- Expiration

Use them for event totals.

Phase 3 — HyperLogLog

Learn:

- Adding unique values
- Cardinality estimation

Use it for unique users.

Phase 4 — Active Users

Define what **active user** means.

For example:

A user who generated an event within the last 5 minutes.

Implement your own logic around this definition.

Phase 5 — Sorted Sets

Use Sorted Sets for:

- Popular pages
 - Most active users
 - Event rankings
-

Phase 6 — MongoDB

Store historical events.

Redis handles fast analytics while MongoDB handles persistent historical records.

Phase 7 — Analytics API

Build the endpoints needed by the dashboard.

Phase 8 — SSE

Create the persistent server-to-browser connection.

Phase 9 — Redis Pub/Sub

Connect Redis updates to the SSE layer.

Phase 10 — Dashboard

Build:

- Overview cards
 - Live activity
 - Popular pages
 - Event charts
-

Phase 11 — Edge Cases and Performance

Investigate:

- Redis failure
 - Duplicate events
 - Expired data
 - Different time windows
 - SSE reconnection
 - Multiple dashboard clients
 - Large numbers of events
 - Concurrent requests
-

Important Design Questions

These are problems to solve yourself while building.

1. What makes a user active?

Define your own activity window.

2. How do you prevent duplicate events?

Consider what happens if the same request is received twice.

3. How do you handle time windows?

Consider:

- Last 5 minutes
- Last hour
- Today
- Yesterday
- Last 7 days

4. How do you identify users?

Think about authenticated users versus anonymous visitors.

5. What happens if Redis goes down?

Decide how your application should behave.

6. What should be stored permanently?

Don't automatically store everything in Redis.

Ask:

Do I need this data permanently, or is it just a real-time aggregate?

V1 Goal

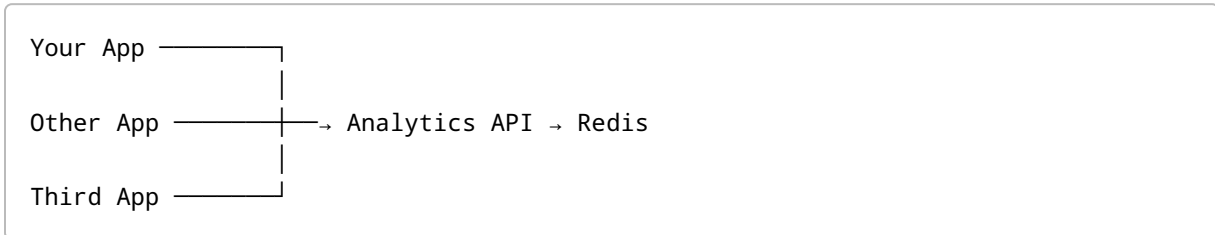
The finished V1 should allow you to:

1. Send user events to the backend.
 2. Process those events with Redis.
 3. Count events.
 4. Estimate unique users with HyperLogLog.
 5. Rank pages with Sorted Sets.
 6. Store historical events in MongoDB.
 7. Stream analytics updates through SSE.
 8. Display the results on a live dashboard.
-

Future Version

After V1 works, turn it into a standalone analytics service.

Instead of only your own application sending events:



Eventually, another application could integrate with your analytics platform and send its own events.

This would turn the project from a simple analytics dashboard into a small **analytics platform/API**.